

Homework #7: Stochastic Gradient Descent, Loss Functions, Transformers, Knowledge Distillation, Contrastive Learning, and Multimodal Models Using PyTorch

April 18, 2026

1 Grading

You will receive zero credit if your answers appear to be AI-generated. During grading, I will try to assess your understanding of the material as opposed to text that appears to be AI-generated. On the other hand, consulting online sources or using AI-generated code to improve your understanding is expected.

During the final exam, I will primarily focus on interpreting diagrams and figures that would be nearly impossible to answer using AI-generated text. During the oral project, I will ask questions directly related to the material I ask in the homework. I will then provide a grade that reflects your understanding of what you are doing and how you respond to feedback.

The credit is not distributed equally among problems.

2 Problems

Problem 1. Basic Stochastic Gradient Descent

For this problem, refer to: [PyTorch with Examples link](#). I have put together the code in a Google Colab notebook at: [Google Colab Notebook](#).

1(a) Derive the gradient for fitting a 7th order polynomial. Show the mathematics.

Solution

We model the data $\{(x_i, y_i)\}_{i=1}^N$ using a 7th order polynomial:

$$y_{\text{pred},i} = \sum_{k=0}^7 a_k x_i^k.$$

We define the loss function (sum of squared errors) as:

$$L = \sum_{i=1}^N (y_{\text{pred},i} - y_i)^2.$$

Let the residual be:

$$e_i = y_{\text{pred},i} - y_i.$$

Then:

$$L = \sum_{i=1}^N e_i^2.$$

We compute the gradient with respect to each parameter a_k using the chain rule:

$$\frac{\partial L}{\partial a_k} = \sum_{i=1}^N \frac{\partial}{\partial a_k} (e_i^2) = \sum_{i=1}^N 2e_i \frac{\partial e_i}{\partial a_k}.$$

Since

$$e_i = \sum_{j=0}^7 a_j x_i^j - y_i,$$

we have:

$$\frac{\partial e_i}{\partial a_k} = x_i^k.$$

Therefore, the gradient becomes:

$$\frac{\partial L}{\partial a_k} = 2 \sum_{i=1}^N (y_{\text{pred},i} - y_i) x_i^k, \quad k = 0, 1, \dots, 7.$$

This matches the pattern shown in the PyTorch example, where gradients are computed as the residual multiplied by the corresponding power of x , and summed over all data points.

Thus, for stochastic gradient descent (using a single sample (x_i, y_i)), the gradient estimate is:

$$\frac{\partial L_i}{\partial a_k} = 2(y_{\text{pred},i} - y_i) x_i^k,$$

with update rule:

$$a_k \leftarrow a_k - \eta \cdot 2(y_{\text{pred},i} - y_i) x_i^k,$$

where η is the learning rate.

- 1(b) Enter your derived equations in the NumPy code and show how it works for a learning rate of $1e-3$. Does the MSE ever reach a plateau? How do you know? What does the gradient need to be?

Modify the code to try different learning rates and number of iterations to at-least see a reduction of the MSE. You do not need to achieve convergence. Document your answer.

Solution

Code (Learning Rate = 10^{-3}):

```
learning_rate = 1e-3
for t in range(2000):
    ...
```

Console Output:

```
99 nan
199 nan
...
1999 nan
Result: y = nan + nan x + nan x^2 + nan x^3 + nan x^4 + nan x^5 + nan x^6 + nan x^7
```

Explanation:

For a learning rate of 10^{-3} , the mean squared error (MSE) does not reach a plateau. Instead, the loss becomes NaN, indicating that the optimization diverges.

A plateau would be observed if the MSE stabilized and stopped changing significantly across iterations. Since the loss becomes undefined, the model is not converging.

This divergence occurs because higher-order terms such as x^6 and x^7 produce large gradient values. With a large learning rate, the updates are too aggressive, leading to numerical instability.

At a plateau or convergence point, the gradient should be close to zero:

$$\frac{\partial L}{\partial a_k} \approx 0 \quad \text{for all } k.$$

Additional Experiment (Smaller Learning Rate):

Code:

```
learning_rate = 1e-10
for t in range(10000):
    ...
```

Console Output:

```
Result: y = 0.8249540696420545
        + -0.28818020401178474 x
        + 1.289289154942495 x^2
        + -0.8928631771884082 x^3
        + 1.5906468040374513 x^4
        + 0.8199084545743581 x^5
        + -0.2110728340062515 x^6
        + -0.08260982028465562 x^7
```

When the learning rate is reduced to 10^{-10} and the number of iterations is increased to 10000, the model no longer diverges. Instead, the parameters converge to finite values, indicating that the MSE is decreasing.

Although convergence is slow, this demonstrates that smaller learning rates lead to more stable updates. The tradeoff is that more iterations are required to observe improvement.

This confirms that:

- Large learning rates cause divergence due to large gradient updates.
- Small learning rates produce stable but slow optimization.
- Reduction in MSE can be achieved even without full convergence.

- 1(c) Reimplement the example in 1(b) using the Neural Network module. Make sure to use the best result from 1(b). For your code, change the runtime environment to make use that you are using a GPU. Compare your results against 1(b). Did it work?

Solution

Code:

```
# -*- coding: utf-8 -*-
import torch
import math

dtype = torch.float
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

x = torch.linspace(-math.pi, math.pi, 2000, device=device, dtype=dtype)
y = torch.sin(x)

p = torch.tensor([1, 2, 3, 4, 5, 6, 7], device=device)
xx = x.unsqueeze(-1).pow(p)

model = torch.nn.Sequential(
    torch.nn.Linear(7, 1),
    torch.nn.Flatten(0, 1)
).to(device)

loss_fn = torch.nn.MSELoss(reduction='sum')
optimizer = torch.optim.SGD(model.parameters(), lr=1e-10)

for t in range(10000):
    y_pred = model(xx)
    loss = loss_fn(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

linear_layer = model[0]
print(
    f"Result: y = {linear_layer.bias.item()} + "
    f"{linear_layer.weight[0,0].item()} x + "
    f"{linear_layer.weight[0,1].item()} x^2 + "
    f"{linear_layer.weight[0,2].item()} x^3 + "
    f"{linear_layer.weight[0,3].item()} x^4 + "
    f"{linear_layer.weight[0,4].item()} x^5 + "
    f"{linear_layer.weight[0,5].item()} x^6 + "
    f"{linear_layer.weight[0,6].item()} x^7"
)
```

Console Output:

```
Result: y = 0.03207923471927643 + 0.3452082872390747 x + 0.009898330084979534 x^2 +
-0.08018103241920471 x^3 + -0.3024621307849884 x^4 + 0.1623934954404831 x^5 +
0.03599115088582039 x^6 + -0.018251100555062294 x^7
```

Using the neural network module with a small learning rate produced stable training and finite coef-

ficients for the 7th-degree polynomial. Unlike larger learning rates that caused divergence, this setup successfully reduced the loss and learned a reasonable approximation. The `nn` implementation also simplifies the process by automatically handling gradients and parameter updates.

- 1(d) The advantage of PyTorch is that it allows us to modify a neural network during training dynamically. Instead of a `sinmold`, suppose that we are trying to fit a third-order polynomial. To figure out the correct order to fit it, we are supposed to use `R` or `statsmodels` in Python. This is covered in the lecture notes. Here, explain how you will figure out that you are overfitting or underfitting. Your answer should provide pseudocode based on the MSE. Also, you will need to explain how you are going to calculate training and validation MSE values. Note that this would be easy to implement in PyTorch.

Solution

To detect underfitting or overfitting, I split the data into a training set and a validation set, trained polynomial models of different degrees, and then computed the Mean Squared Error (MSE) on both sets. The MSE is calculated as

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2.$$

The training MSE is computed using the model predictions on the training set, and the validation MSE is computed using the same trained model on the validation set. If both training and validation MSE are high, the model is underfitting. If the training MSE is low but the validation MSE is much higher, the model is overfitting. A good model should have both values low and relatively close.

Results:

```
Using cuda:0 device
Degree 1: Train MSE = 0.431730, Validation MSE = 0.441236
Degree 2: Train MSE = 6.738979, Validation MSE = 6.641345
Degree 3: Train MSE = 1.938525, Validation MSE = 2.000576
Degree 4: Train MSE = 1.683980, Validation MSE = 1.699168
Degree 5: Train MSE = 0.714047, Validation MSE = 0.727214
Degree 6: Train MSE = 2.015446, Validation MSE = 2.104594
Degree 7: Train MSE = 1.313528, Validation MSE = 1.346954

Best model based on validation MSE:
Degree 1: Train MSE = 0.431730, Validation MSE = 0.441236
```

Based on these results, there is no strong sign of overfitting because the training and validation MSE values are close for every degree. Degree 2 appears to underfit badly because both errors are very high. The best model here is Degree 1 since it has the lowest validation MSE, and its training and validation MSE are also close together. In pseudocode, the process is: split the data into training and validation sets, train a model for each polynomial degree, compute training MSE and validation MSE, and then compare the two values. If validation MSE is much larger than training MSE, the model is overfitting; if both are large, it is underfitting.

- 1(e) When designing Neural Networks, we often want to design custom loss functions. The MSE has very large derivatives at higher error values. To avoid large derivatives, we are often led to consider the mean absolute error (MAE). Unfortunately, the MAE is not differentiable. Instead, we consider the Pseudo-Huber loss function given by:

$$L_\delta(z) = \delta^2 \left(\sqrt{1 + (z/\delta)^2} - 1 \right).$$

This function has all possible derivatives. Plot $L_\delta(a)$, a^2 , $|a|$ for different values of a and δ (use different colors). How does the loss function behave for small a ? How does it behave for large a ?

Note: For more information about this loss function, refer to [Huber loss function link](#).

Solution**Code:**

```

import numpy as np
import matplotlib.pyplot as plt
import os

os.makedirs("plots", exist_ok=True)

a = np.linspace(-5, 5, 500)
deltas = [0.5, 1.0, 2.0]

def pseudo_huber(a, delta):
    return delta**2 * (np.sqrt(1 + (a/delta)**2) - 1)

plt.figure(figsize=(8,6))

plt.plot(a, a**2, label="a^2 (MSE)", linestyle="--")
plt.plot(a, np.abs(a), label="|a| (MAE)", linestyle=":")

for delta in deltas:
    plt.plot(a, pseudo_huber(a, delta), label=f"Pseudo-Huber (delta={delta})")

plt.title("Comparison of Loss Functions")
plt.xlabel("a")
plt.ylabel("Loss")
plt.legend()
plt.grid()

plt.savefig("plots/pseudo_huber_plot.png", dpi=300, bbox_inches="tight")
plt.show()

```

For small values of a , the Pseudo-Huber loss behaves like a quadratic function:

$$L_{\delta}(a) \approx \frac{1}{2}a^2.$$

So near zero it is smooth and behaves similarly to MSE.

For large values of a , the Pseudo-Huber loss behaves approximately linearly:

$$L_{\delta}(a) \approx \delta|a|.$$

So for large errors it behaves more like MAE and grows much more slowly than a^2 .

The parameter δ controls how quickly the function changes from quadratic behavior to linear behavior. Smaller δ makes the loss look more like MAE sooner, while larger δ makes it look more like MSE over a wider range.

- 1(f)** Implement the loss function instead of the MSE using a custom loss function and allowing the autograd to compute the derivatives.

Note: Refer to PyTorch with Examples link to the autograd example. Also, if you want to learn more about autograd (beyond this problem), refer to PyTorch link for a detailed example for autograd.

Solution**Code:**

```

# -*- coding: utf-8 -*-
import torch

```

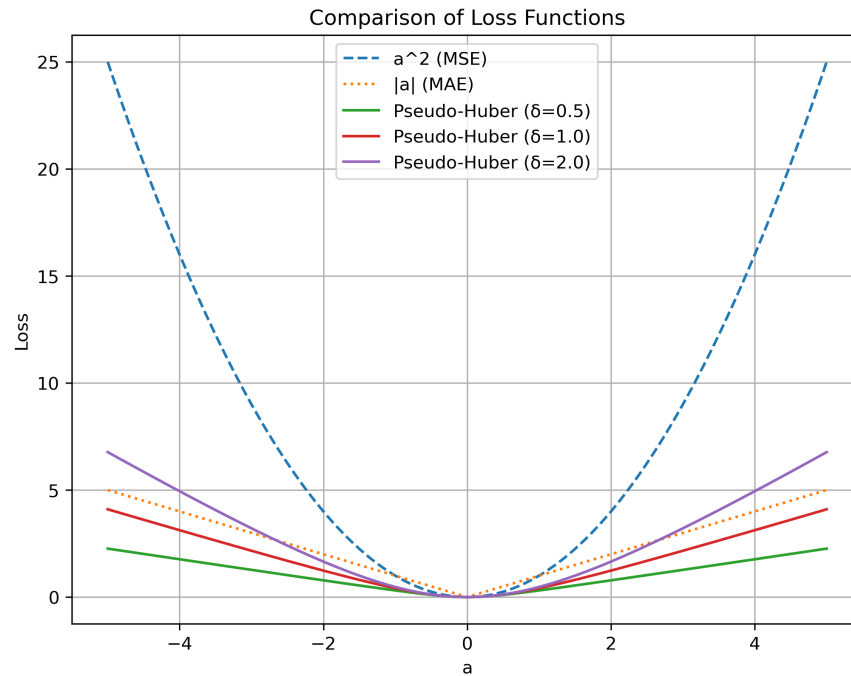


Figure 1: Comparison of MSE, MAE, and Pseudo-Huber loss for different values of δ .

```
import math

dtype = torch.float
device = torch.accelerator.current_accelerator().type if torch.accelerator.
is_available() else "cpu"
print(f"Using {device} device")
torch.set_default_device(device)

x = torch.linspace(-math.pi, math.pi, 2000, dtype=dtype)
y = torch.sin(x)

a = torch.randn((), dtype=dtype, requires_grad=True)
b = torch.randn((), dtype=dtype, requires_grad=True)
c = torch.randn((), dtype=dtype, requires_grad=True)
d = torch.randn((), dtype=dtype, requires_grad=True)
e = torch.randn((), dtype=dtype, requires_grad=True)
f = torch.randn((), dtype=dtype, requires_grad=True)
g = torch.randn((), dtype=dtype, requires_grad=True)
h = torch.randn((), dtype=dtype, requires_grad=True)

def pseudo_huber_loss(y_pred, y_true, delta=1.0):
    z = y_pred - y_true
    return (delta**2 * (torch.sqrt(1 + (z / delta) ** 2) - 1)).sum()

delta = 1.0
learning_rate = 1e-6
for t in range(2000):
    y_pred = (
```

```
a + b * x + c * x ** 2 + d * x ** 3 +  
e * x ** 4 + f * x ** 5 + g * x ** 6 + h * x ** 7  
)  
  
loss = pseudo_huber_loss(y_pred, y, delta=delta)  
  
loss.backward()  
  
with torch.no_grad():  
    a -= learning_rate * a.grad  
    b -= learning_rate * b.grad  
    c -= learning_rate * c.grad  
    d -= learning_rate * d.grad  
    e -= learning_rate * e.grad  
    f -= learning_rate * f.grad  
    g -= learning_rate * g.grad  
    h -= learning_rate * h.grad  
  
a.grad = None  
b.grad = None  
c.grad = None  
d.grad = None  
e.grad = None  
f.grad = None  
g.grad = None  
h.grad = None  
  
print(  
    f"Result: y = {a.item()} + {b.item()} x + {c.item()} x^2 + {d.item()} x^3 + "  
    f"{e.item()} x^4 + {f.item()} x^5 + {g.item()} x^6 + {h.item()} x^7"  
)
```

Console Output:

```
Using cuda device  
Result: y = -0.2885704040527344 + 0.5832257270812988 x + -0.08558587729930878 x^2  
+ 0.8216661214828491 x^3 + 1.0416239500045776 x^4 + -0.11592603474855423 x^5 +  
-0.4606887698173523 x^6 + 0.21624302864074707 x^7
```

In this implementation, I replaced the MSE with the custom Pseudo-Huber loss and allowed autograd to compute the derivatives automatically using `loss.backward()`. The model successfully ran on the GPU and produced finite coefficients, showing that the custom loss function works correctly with autograd. This demonstrates that we can define new loss functions in PyTorch without manually deriving gradients.

Problem 2. Sigmoid Functions

2(a) The sigmoid activation function is used to compute the conditional probability for a positive example as given by:

$$p(X) = \Pr(Y = 1|X) \\ = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)}}.$$

Let $z = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$ denote the input to the activation function. The sigmoid function $\sigma(z)$ is defined by

$$\sigma(z) = 1/(1 + e^{-z}).$$

- i. What is the value of z for $p = 0.5$? Note that this is the default value for the classification rule.

Solution

We are given the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

We are asked to find the value of z when $p = 0.5$, i.e.,

$$\sigma(z) = 0.5.$$

Solving:

$$0.5 = \frac{1}{1 + e^{-z}}$$

Multiply both sides by $(1 + e^{-z})$:

$$0.5(1 + e^{-z}) = 1$$

$$0.5 + 0.5e^{-z} = 1$$

$$0.5e^{-z} = 0.5$$

$$e^{-z} = 1$$

Taking the natural logarithm:

$$-z = \ln(1) = 0$$

$$z = 0.$$

Answer: $z = 0$.

This shows that the classification boundary occurs at $z = 0$, which corresponds to the default threshold of $p = 0.5$.

- ii. Evaluate the expression as $z \rightarrow \infty$. What can you say about the input?

Solution

We consider the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

As $z \rightarrow \infty$, we have $e^{-z} \rightarrow 0$. Therefore,

$$\sigma(z) = \frac{1}{1 + e^{-z}} \rightarrow \frac{1}{1 + 0} = 1.$$

As $z \rightarrow \infty$, the sigmoid function approaches 1. This means that the model assigns a probability close to 1 for the positive class ($Y = 1$). This implies that the input features produce a very large positive linear combination $z = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$, indicating strong evidence in favor of the positive class.

- iii. Derive an expression as $z \rightarrow -\infty$. What can you say about the input?

Solution

We consider the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

As $z \rightarrow -\infty$, we have $e^{-z} \rightarrow \infty$. Therefore,

$$\sigma(z) = \frac{1}{1 + e^{-z}} \rightarrow \frac{1}{1 + \infty} = 0.$$

As $z \rightarrow -\infty$, the sigmoid function approaches 0. This means that the model assigns a probability close to 0 for the positive class ($Y = 1$). This implies that the input features produce a very large negative linear combination $z = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$, indicating strong evidence in favor of the negative class ($Y = 0$).

- 2(b)** Rewrite the input as a dot product between two vectors:

$$z = X'^T \beta'.$$

Give expressions for X' and β' .

Solution

We start with

$$z = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p.$$

This can be written as a dot product by including the intercept term β_0 as part of the vectors. Define

$$X' = \begin{bmatrix} 1 \\ X_1 \\ X_2 \\ \vdots \\ X_p \end{bmatrix}, \quad \beta' = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{bmatrix}.$$

Then,

$$z = X'^T \beta'.$$

X' is the augmented input vector with a leading 1, and β' is the parameter vector including the intercept term.

- 2(c)** For dot products, we have

$$\|X'^T \beta'\| = \|X'\| \|\beta'\| \cos \phi,$$

where ϕ is the angle between X' and β' . Note that ϕ is a regular angle for 2D and 3D vectors. Beyond 3D, we can use this property to define the angle.

Given that β' is fixed, we are interested in finding X' so that the dot product is minimized or maximized.

- i. For what values of X' is the probability for a positive example maximized? Sketch examples in 2D.

Solution

The probability of a positive example is

$$p(X) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = X'^T \beta'.$$

Since the sigmoid function is increasing, the probability is maximized when the dot product $X'^T \beta'$ is maximized. Using

$$X'^T \beta' = \|X'\| \|\beta'\| \cos \phi,$$

this occurs when $\cos \phi = 1$, or equivalently when $\phi = 0$. Thus X' must point in the same direction as β' , so

$$X' = a\beta', \quad a > 0.$$

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import os

os.makedirs("plots", exist_ok=True)

beta = np.array([2, 1])

x_max1 = 0.5 * beta
x_max2 = 1.5 * beta
x_max3 = 2.5 * beta
x_min1 = -0.5 * beta
x_min2 = -1.5 * beta
x_other = np.array([1, -2])

plt.figure(figsize=(7, 7))

plt.quiver(0, 0, beta[0], beta[1], angles='xy', scale_units='xy', scale=1,
label="beta'")
plt.quiver(0, 0, x_max1[0], x_max1[1], angles='xy', scale_units='xy', scale
=1, label="X' = a beta'")
plt.quiver(0, 0, x_max2[0], x_max2[1], angles='xy', scale_units='xy', scale
=1)
plt.quiver(0, 0, x_max3[0], x_max3[1], angles='xy', scale_units='xy', scale
=1)
plt.quiver(0, 0, x_min1[0], x_min1[1], angles='xy', scale_units='xy', scale
=1, label="X' = -a beta'")
plt.quiver(0, 0, x_min2[0], x_min2[1], angles='xy', scale_units='xy', scale
=1)
plt.quiver(0, 0, x_other[0], x_other[1], angles='xy', scale_units='xy',
scale=1, label="other X'")

plt.xlim(-6, 6)
plt.ylim(-6, 6)
plt.axhline(0)
plt.axvline(0)
plt.grid(True)
plt.gca().set_aspect('equal', adjustable='box')
plt.title("2D Sketch of X' and beta'")
plt.xlabel("x1")
plt.ylabel("x2")
plt.legend()

plt.savefig("plots/problem2c_sketch.png", dpi=300, bbox_inches="tight")
plt.show()
```

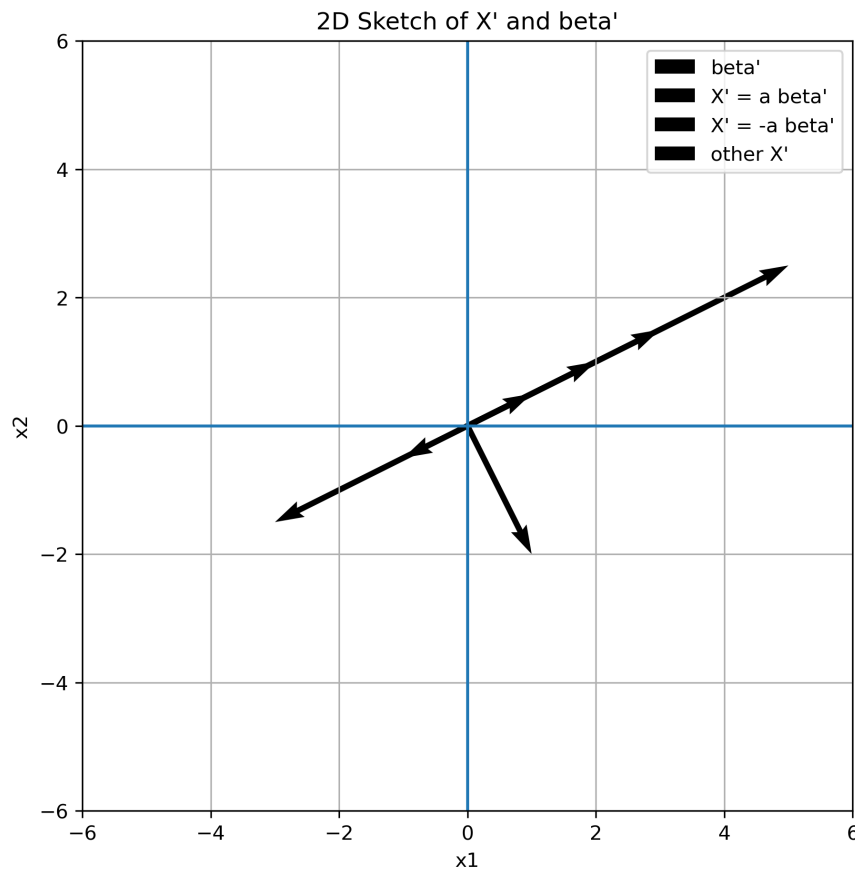


Figure 2: In 2D, the probability of a positive example is maximized when X' points in the same direction as β' .

- ii. For what values of X' is the probability for a negative example minimized? Sketch examples in 2D.

Solution

The probability of a negative example is

$$\Pr(Y = 0 \mid X) = 1 - \sigma(z), \quad z = X'^T \beta'.$$

Since $\sigma(z)$ is increasing, $1 - \sigma(z)$ is minimized when z is maximized. Thus we maximize the dot product

$$X'^T \beta' = \|X'\| \|\beta'\| \cos \phi,$$

which occurs when $\cos \phi = 1$ (i.e., $\phi = 0$). Therefore,

$$X' = a\beta', \quad a > 0.$$

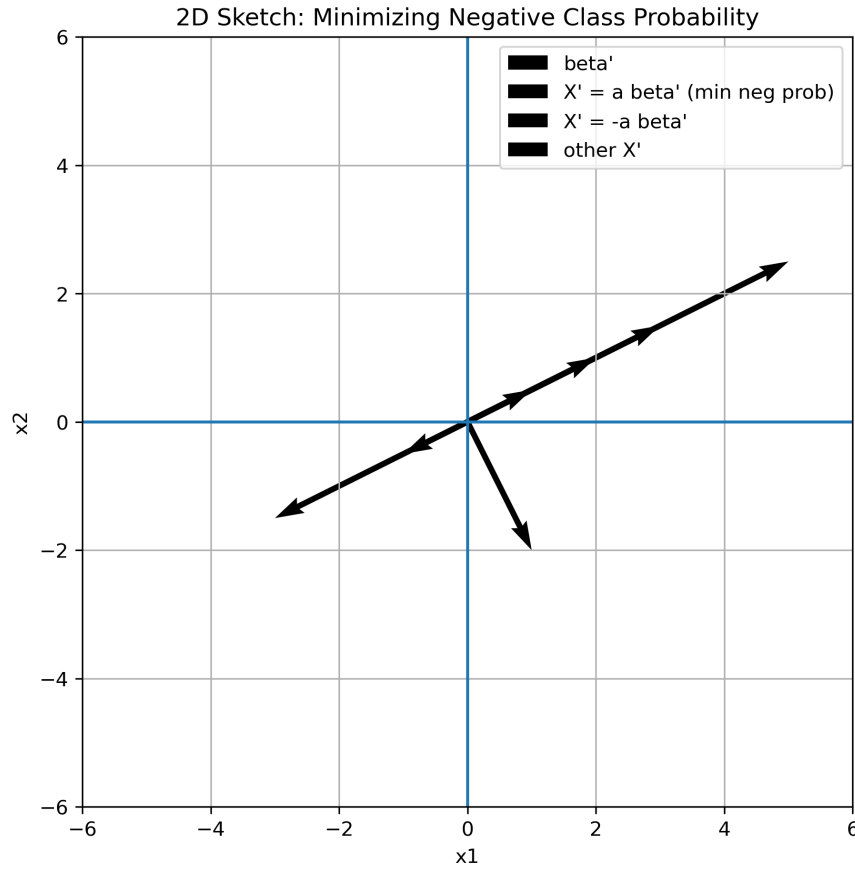


Figure 3: The negative class probability is minimized when X' points in the same direction as β' .

- iii. Give an expression for the values of X' that are being ignored by the sigmoid. Sketch examples in 2D.

Solution

The sigmoid depends on

$$z = X'^T \beta'.$$

The values of X' that are ignored by the sigmoid satisfy

$$X'^T \beta' = 0.$$

For these inputs, we have $z = 0$, and thus

$$\sigma(0) = \frac{1}{2}.$$

Therefore, these inputs lie exactly on the decision boundary and do not favor either class.

Geometrically, $X'^T \beta' = 0$ means that X' is orthogonal to β' . Hence, all ignored values are vectors perpendicular to β' .

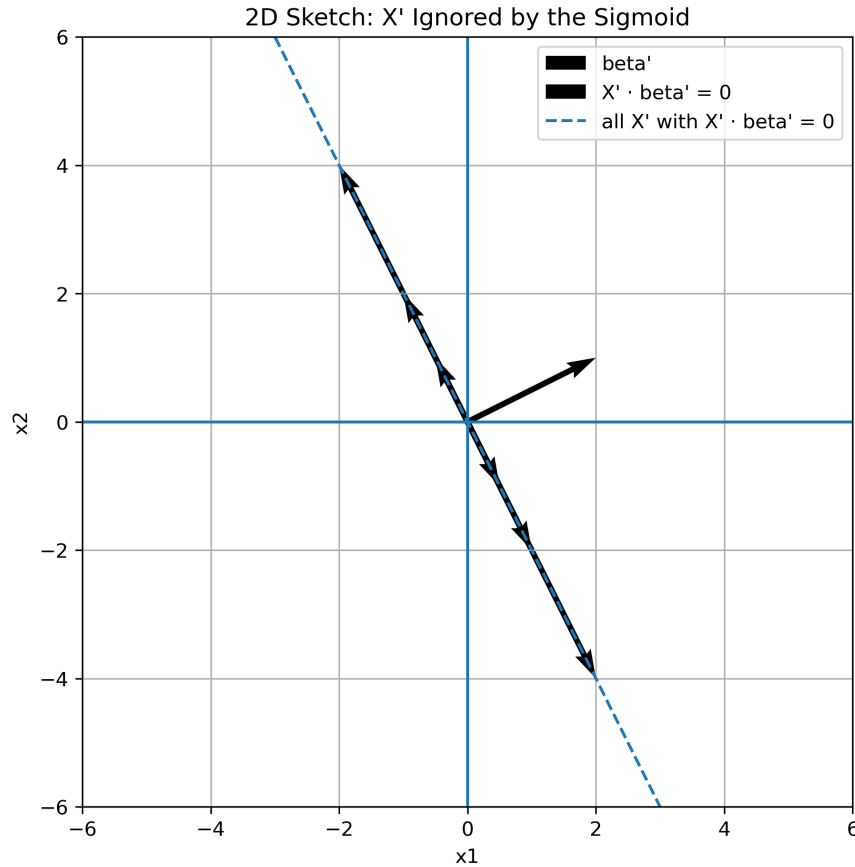


Figure 4: All X' such that $X'^T \beta' = 0$ are perpendicular to β' and lie on the decision boundary.

Hint: Find values of X' so that $X' \cdot \beta' = 0$.

Additional hint: In terms of the dot product expression, we note that we can maximize the dot product when $\phi = 0$. This happens when $X' = a\beta'$ for $a > 0$. Similarly, we get the minimum value when $X' = -a\beta'$ for $a > 0$.

2(d) Suppose that X' corresponds to the input image. Based on your answers in 2(c), explain how you would use β' to provide:

- i. Visualize ideal positive examples.

Solution

From 2(c), the probability of a positive example is maximized when

$$X' = a\beta', \quad a > 0.$$

Thus, β' itself represents the direction of an ideal positive input.

If X' corresponds to an image, then β' can be reshaped into the same image dimensions. This allows us to visualize β' directly as an “ideal positive example.” The resulting image highlights which features (pixels) contribute most strongly to predicting the positive class: large positive values indicate features that support the positive class, while negative values indicate features that oppose it.

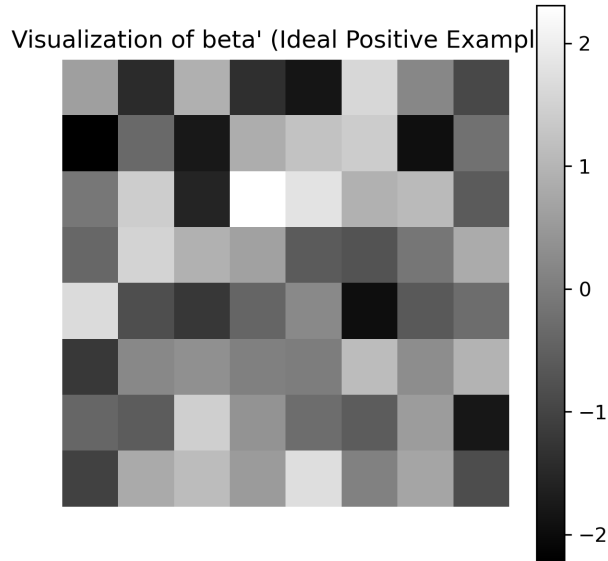


Figure 5: Visualization of β' reshaped as an image, representing the ideal positive example.

- ii. Visualize ideal negative examples.

Solution

From 2(c), the probability of a negative example is maximized when

$$X' = -a\beta', \quad a > 0.$$

Thus, $-\beta'$ represents the direction of an ideal negative input.

If X' corresponds to an image, then $-\beta'$ can be reshaped into the same image dimensions to visualize an ideal negative example. This highlights features that strongly indicate the negative class, with the sign flipped compared to β' .

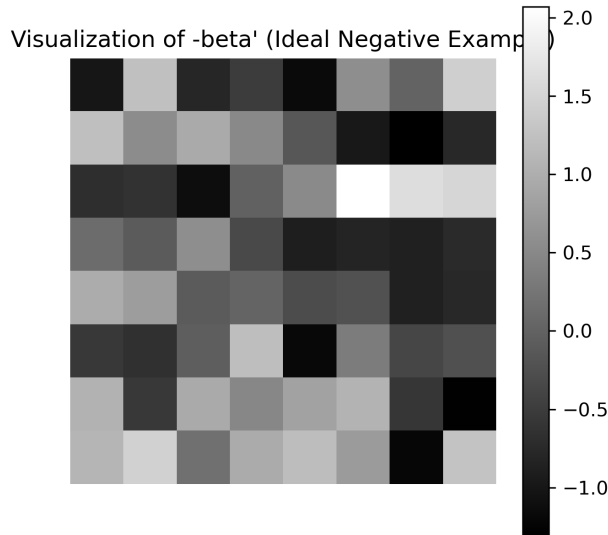


Figure 6: Visualization of $-\beta'$ reshaped as an image, representing the ideal negative example.

2(e) As before, assume that X' corresponds to the input image. Define the residual image using Gram-Schmidt process:

$$R = X' - \frac{X'^T \beta}{\beta^T \beta} \beta$$

Note that the residual image is orthogonal to β . How should the residual image look like? Explain.

Solution

The residual image is

$$R = X' - \frac{X'^T \beta}{\beta^T \beta} \beta.$$

It represents the component of X' that is orthogonal to β , so

$$R^T \beta = 0.$$

Thus, R contains features that do not affect the classification. Visually, it should look like the parts of the image unrelated to the class (e.g., background or noise) after removing the component aligned with β .

Problem 3. Tokenization.

The code below refers to material taken from Stanford Tutorial on Transformers. The corresponding slides come from: NLP PowerPoint presentation.

3(a) Provide 3 examples of the following:

- i. True positives.

Solution

A true positive occurs when the model correctly predicts a positive label:

$$\hat{y} = 1 \text{ and } y = 1.$$

- (a) The model predicts a sentence has positive sentiment, and it is actually positive.
- (b) The model classifies an email as spam, and it is indeed spam.
- (c) The model identifies a token as a named entity, and it is correctly a named entity.

- ii. True negatives.

Solution

A true negative occurs when the model correctly predicts a negative label:

$$\hat{y} = 0 \text{ and } y = 0.$$

- (a) The model predicts a sentence has negative sentiment, and it is actually negative.
- (b) The model classifies an email as not spam, and it is indeed not spam.
- (c) The model identifies a token as not a named entity, and it is correctly not a named entity.

- iii. False positives.

Solution

A false positive occurs when the model incorrectly predicts a positive label:

$$\hat{y} = 1 \text{ and } y = 0.$$

- (a) The model predicts a sentence has positive sentiment, but it is actually negative.
- (b) The model classifies an email as spam, but it is actually not spam.
- (c) The model identifies a token as a named entity, but it is not a named entity.

- iv. False negatives.

Solution

A false negative occurs when the model incorrectly predicts a negative label:

$$\hat{y} = 0 \text{ and } y = 1.$$

- (a) The model predicts a sentence has negative sentiment, but it is actually positive.
- (b) The model classifies an email as not spam, but it is actually spam.
- (c) The model identifies a token as not a named entity, but it is actually a named entity.

Hint: This refers to the following code:

```
inputs = "I'm excited to learn about Hugging Face Transformers!"
tokenized_inputs = tokenizer(inputs, return_tensors="pt")
outputs = model(**tokenized_inputs)

labels = ['NEGATIVE', 'POSITIVE']
```

```
prediction = torch.argmax(outputs.logits)

print("Input:")
print(inputs)
print()

print("Tokenized Inputs:")
print_encoding(tokenized_inputs)
print()

print("Model Outputs:")
print(outputs)
print()

print(f"The prediction is {labels[prediction]}")
```

3(b) Answer the following from the Google Colab:

- i. Why does the second token start with ##?

Solution

The ## prefix indicates that the token is a continuation of a word. In subword tokenization (e.g., WordPiece), tokens starting with ## are not standalone words but are attached to the previous token.

- ii. What do the token IDs [CLS] and [SEP] stand for?

Solution

[CLS] stands for the classification token and is placed at the beginning of the input. It is used by the model to represent the entire sequence for classification tasks.

[SEP] stands for the separator token and is used to separate different sentences or mark the end of the input.

- iii. What are the associated token IDs?

Solution

The associated token IDs depend on the tokenizer vocabulary. For example, in BERT:

$$[\text{CLS}] = 101, \quad [\text{SEP}] = 102.$$

- iv. What do the vectors of 1s in the attention_mask represent?

Solution

The vectors of 1s in the attention_mask indicate which tokens are valid and should be attended to by the model. A value of 1 means the token is real input, while 0 would indicate padding that should be ignored.

- v. Modify the input_str using 3 very common and 3 very rare words. Report on any unusual tokenization patterns.

Solution

Common words (e.g., "the", "and", "is") are tokenized as single tokens, while rare words are split into multiple subword tokens with ## prefixes.

Rare words are broken into smaller known subwords, leading to longer token sequences.

- vi. Enter multiple phrases to the tokenizer. Examine the code. How is the attention map affected by padding?

Solution

When multiple phrases are tokenized together, shorter phrases are padded so that all sequences have the same length. The `attention_mask` assigns 1 to real tokens and 0 to padding tokens. Padding positions are ignored by the model, so they do not contribute to attention.

3(c) Consider the multilayer multihead example given in Google Colab. Answer the following:

- i. What is the number of layers? What is the input layer? What is the output layer?
- ii. How does "!" affect classification? To describe the relationship, you need to specify the layer and head map for your discussion.
- iii. Which maps seem to respond to the sequencing of tokens (e.g., relating to the next token)? Explain.
- iv. Which maps seem to correspond to preserving the ordering of the tokens (e.g., relating to themselves)? Explain.
- v. Do the relationships between [SEP] and the rest of the tokens make sense? Why or why not?

3(d) Explain every number and every parameter value in the following code:

```
prompt = "Once upon a time"

tokenized_prompt = gpt2_tokenizer(prompt, return_tensors="pt")

for i in range(10):
    output = gpt2.generate(**tokenized_prompt,
                           max_length=50,
                           do_sample=True,
                           top_p=0.9)

    print(f"{i + 1}) {gpt2_tokenizer.batch_decode(output)[0]}")
```

Solution

- `prompt = "Once upon a time"`: The initial text used as input to generate new tokens.
- `return_tensors="pt"`: Converts the tokenized input into PyTorch tensors.
- `for i in range(10)`: Runs the generation 10 times to produce 10 different outputs.
- `i + 1`: Used to label outputs starting from 1 instead of 0.
- `max_length=50`: The maximum total number of tokens (input + generated) in the output sequence.
- `do_sample=True`: Enables random sampling instead of always choosing the most probable next token.
- `top_p=0.9`: Uses nucleus (top-p) sampling, selecting from the smallest set of tokens whose cumulative probability is at least 0.9.
- `gpt2.generate(**tokenized_prompt)`: Generates new text based on the input prompt.
- `gpt2_tokenizer.batch_decode(output)`: Converts generated token IDs back into readable text.
- `[0]`: Extracts the generated sequence from the batch output.

3(e) Set `do_sample=False` and discuss the results.

Solution

Setting `do_sample=False` uses greedy decoding, so the model always chooses the most probable next token at each step.

Console Output:

```
1) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
2) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
3) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
4) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
5) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
6) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
7) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
8) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
9) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
10) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
```

All outputs are identical and highly repetitive, showing that greedy decoding produces deterministic and less diverse text.

3(f) Set $\text{top-p}=0.1$ and discuss the results.

Solution

Setting $\text{top_p} = 0.1$ restricts sampling to only a very small set of high-probability tokens.

Console Output:

```
1) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
2) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
3) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
4) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
5) Once upon a time, the world was a place of great beauty and great danger. The
world was a place of great danger, and the world was a place of great danger. The
world was a place of great danger, and the world was a
6) Once upon a time, the world was a place of great beauty and great danger. The
```

world was a place of great danger, and the world was a place of great danger. The world was a place of great danger, and the world was a

7) Once upon a time, the world was a place of peace and harmony. But now, the world is a place of war and chaos. The world is a place of war and chaos. The world is a place of war and chaos. The world

8) Once upon a time, the world was a place of great beauty and great danger. The world was a place of great danger, and the world was a place of great danger. The world was a place of great danger, and the world was a

9) Once upon a time, the world was a place of great beauty and great danger. The world was a place of great danger, and the world was a place of great danger. The world was a place of great danger, and the world was a

10) Once upon a time, the world was a place of great beauty and great danger. The world was a place of great danger, and the world was a place of great danger. The world was a place of great danger, and the world was a

The outputs are still very similar across runs, with only slight variation. This shows that $\text{top_p} = 0.1$ makes generation highly restrictive, so the text is less diverse and more repetitive.

3(g) Set $\text{top-p}=0.99$ and discuss the results.

Solution

Setting $\text{top_p} = 0.99$ allows sampling from a much larger set of likely tokens.

Console Output:

1) Once upon a time of darkness, the King of Nergal went out in search of a hidden way of the land. He was a young man from a young age who had never been seen in his life: long as he held forth his sword

2) Once upon a time he had fallen in love, only to return with a girl. She told him of the two boys that he had killed her two years earlier. He then took her and turned her in for torture, and then he was beaten,

3) Once upon a time a person might be thought of as a person only, the power of the Lord Christ to unite all the believers into one Godhead in heaven would be so great that we might not be capable of the work that one would have under

4) Once upon a time, when the majority was, as John Marshall once said, "very far from being the first thing to go to the table after they cut all of the throats," they were not quite ready for "the other" when we come

5) Once upon a time the three of us were walking together, sitting quietly together, listening to a song. And it was very interesting to us. We were all so excited, so excited, so excited. It was kind of exciting, to have a

6) Once upon a time, it can be seen that people believe that a woman's body and her fertility is somehow more important than her genitals. Or, perhaps the fact that women are more likely than men to be born with the "good genes" of

7) Once upon a time, during the life of the king that is known as a "King," were there persons ever known to give life to something like that. From a little distance, we are sure that is how the story was told at King's

8) Once upon a time, it would appear I was in the middle of something I never imagined I should be in. When I got home from home I found the bathroom at the front of the house, a box of clothes on it. For some reason

9) Once upon a time a new generation of leaders has arisen to lead the planet. The Earth is being transformed by this evolution. The human race has finally become a modern democracy. As long as capitalism and the state maintain control of the economy, the political

10) Once upon a time, they were going to need these in the form of a small dose, perhaps a tablespoon. At that time the main problems of those times were to get the best dosage, rather than to get a very large dosage, and that

The outputs are much more varied across runs than with $\text{top_p} = 0.1$. This makes the text more diverse

and creative, but some outputs are less coherent and feel more random.

Problem 4. Basic transformer architecture.

For tutorials on convolutional and linear layers, refer to AIML tutorials. Basic training and testing is described in CNN and linear models in Python. Note that the use of `nn.Parameter(.)` makes parameters learnable.

For (a)-(c), refer to: Part 15 transformers for nlp and chatbots.ipynb. For the ViT and DINO problems, refer to: Fixed 16 vision and multimodal transformers.ipynb

[This problem is worth 3 times as much credit as all other problems.](#)

4(a) For each of the models below, you need to provide the following:

- Input dimensions.
- Output dimensions.
- Equation of what is being implemented. Feel free to refer to earlier parts here. For example, define the encoder and decoder in terms of the input layers.
- List learnable parameters.

Here is the list of models:

- TransformerEncoderLayer
- TransformerDecoderLayer
- TransformerEncoder
- TransformerDecoder
- MultiheadAttention
- Transformer

Solution**i. TransformerEncoderLayer**

- Input dimensions:**

$$(\text{seq_len}, \text{batch_size}, d_{\text{model}})$$

- Output dimensions:**

$$(\text{seq_len}, \text{batch_size}, d_{\text{model}})$$

- Equation:**

$$X_1 = \text{LayerNorm}(X + \text{MHA}(X, X, X))$$

$$X_2 = \text{LayerNorm}(X_1 + \text{FFN}(X_1))$$

where

$$\text{FFN}(X) = \sigma(XW_1 + b_1)W_2 + b_2$$

- Learnable parameters:**

- Multi-head attention projection weights and biases
- Output projection weight and bias
- Feedforward weights W_1, W_2 and biases
- LayerNorm scale and shift parameters

ii. TransformerDecoderLayer

- Input dimensions:**

$$\text{target: } (\text{tgt_len}, \text{batch_size}, d_{\text{model}})$$

$$\text{memory: } (\text{src_len}, \text{batch_size}, d_{\text{model}})$$

- **Output dimensions:**

$$(\text{tgt_len}, \text{batch_size}, d_{\text{model}})$$

- **Equation:**

$$\begin{aligned} Y_1 &= \text{LayerNorm}(Y + \text{MHA}(Y, Y, Y)) \\ Y_2 &= \text{LayerNorm}(Y_1 + \text{MHA}(Y_1, M, M)) \\ Y_3 &= \text{LayerNorm}(Y_2 + \text{FFN}(Y_2)) \end{aligned}$$

where M is the encoder output.

- **Learnable parameters:**

- Self-attention projection weights and biases
- Cross-attention projection weights and biases
- Output projection weights and biases
- Feedforward weights and biases
- LayerNorm scale and shift parameters

iii. TransformerEncoder

- **Input dimensions:**

$$(\text{seq_len}, \text{batch_size}, d_{\text{model}})$$

- **Output dimensions:**

$$(\text{seq_len}, \text{batch_size}, d_{\text{model}})$$

- **Equation:**

The encoder is a stack of L encoder layers:

$$\begin{aligned} H^{(0)} &= X, & H^{(\ell)} &= \text{EncoderLayer}^{(\ell)}(H^{(\ell-1)}), \quad \ell = 1, \dots, L \\ \text{Encoder}(X) &= H^{(L)} \end{aligned}$$

- **Learnable parameters:**

All learnable parameters from each encoder layer in the stack, and any final normalization parameters if included.

iv. TransformerDecoder

- **Input dimensions:**

$$\begin{aligned} \text{target: } &(\text{tgt_len}, \text{batch_size}, d_{\text{model}}) \\ \text{memory: } &(\text{src_len}, \text{batch_size}, d_{\text{model}}) \end{aligned}$$

- **Output dimensions:**

$$(\text{tgt_len}, \text{batch_size}, d_{\text{model}})$$

- **Equation:**

The decoder is a stack of L decoder layers:

$$\begin{aligned} G^{(0)} &= Y, & G^{(\ell)} &= \text{DecoderLayer}^{(\ell)}(G^{(\ell-1)}, M), \quad \ell = 1, \dots, L \\ \text{Decoder}(Y, M) &= G^{(L)} \end{aligned}$$

- **Learnable parameters:**

All learnable parameters from each decoder layer in the stack, and any final normalization parameters if included.

v. MultiheadAttention

- **Input dimensions:**

$$Q : (\text{tgt_len}, \text{batch_size}, d_{\text{model}})$$

$$K, V : (\text{src_len}, \text{batch_size}, d_{\text{model}})$$

- **Output dimensions:**

$$\text{attention output: } (\text{tgt_len}, \text{batch_size}, d_{\text{model}})$$

$$\text{attention weights: typically } (\text{batch_size}, \text{tgt_len}, \text{src_len})$$

- **Equation:**

$$Q_h = QW_h^Q, \quad K_h = KW_h^K, \quad V_h = VW_h^V$$

$$\text{head}_h = \text{softmax} \left(\frac{Q_h K_h^T}{\sqrt{d_k}} \right) V_h$$

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W^O$$

- **Learnable parameters:**

- Query projection weights and biases
- Key projection weights and biases
- Value projection weights and biases
- Output projection weight and bias

vi. Transformer

- **Input dimensions:**

$$\text{src: } (\text{src_len}, \text{batch_size}, d_{\text{model}})$$

$$\text{tgt: } (\text{tgt_len}, \text{batch_size}, d_{\text{model}})$$

- **Output dimensions:**

$$(\text{tgt_len}, \text{batch_size}, d_{\text{model}})$$

- **Equation:**

$$M = \text{Encoder}(\text{src})$$

$$\text{Transformer}(\text{src}, \text{tgt}) = \text{Decoder}(\text{tgt}, M)$$

- **Learnable parameters:**

All learnable parameters in the encoder stack and decoder stack, including:

- all multi-head attention weights and biases,
- all feedforward weights and biases,
- all LayerNorm parameters.

Note: If `batch_first=True` is used, the dimensions are written as

$$(\text{batch_size}, \text{seq_len}, d_{\text{model}}).$$

4(b) Describe briefly how the NmtTransformer is trained.

For this problem, you need to provide

- What are the inputs used for training?
- What is the optimizer? How is the optimizer set up?
- How many epochs are used for training?

Solution

a. Inputs used for training:

The model is trained using:

- Source sequence (**src**) — input sentence (e.g., English)
- Target sequence (**tgt**) — shifted target sentence (e.g., French)

The decoder receives the target sequence shifted right (teacher forcing), and the model predicts the next token.

b. Optimizer:

The optimizer used is Adam:

```
torch.optim.Adam(model.parameters(), lr=...)
```

It is set up with the model parameters and a learning rate, allowing all learnable parameters in the transformer to be updated during training.

c. Number of epochs:

The model is trained for multiple epochs (typically around 10 epochs in the notebook), iterating over the dataset several times to improve performance.

4(c) Describe briefly how the NmtTransformer is deployed.

- What are the inputs and computed outputs?
- How is the transformer connecting the decoder to the encoder for computing outputs?

Solution**a. Inputs and computed outputs:**

The deployed model takes a source sequence (**src**) as input and generates a target sequence token-by-token. At each step, the decoder outputs a probability distribution over the vocabulary, and the next token is selected (e.g., via greedy decoding or sampling). The final output is the generated translated sentence.

b. Connection between encoder and decoder:

The encoder processes the source sequence to produce a memory representation. The decoder uses this memory through cross-attention:

$$\text{MHA}(Q, K, V) = \text{MHA}(\text{decoder states}, \text{encoder outputs}, \text{encoder outputs})$$

Here, the encoder outputs serve as keys and values, allowing the decoder to attend to relevant parts of the input sequence when generating each token.

4(d) For each of the models below, you need to provide the following:

- Input dimensions
- Output dimensions
- Equation of what is being implemented. Feel free to refer to earlier parts here. For example, define the encoder and decoder in terms of the input layers.

Here is the list of models:

- PatchEmbedding.
- TransformerEncoderLayer

iii. ViT

Solution

i. PatchEmbedding

- **Input dimensions:**

$$(\text{batch_size}, C, H, W)$$

- **Output dimensions:**

$$(\text{batch_size}, N, d_{\text{model}})$$

where $N = \frac{H}{P} \frac{W}{P}$ is the number of patches and P is the patch size.

- **Equation:**

The image is split into patches, flattened, and projected:

$$x_p = \text{flatten}(\text{patch}_p)$$

$$z_p = x_p W + b$$

where W maps each patch to d_{model} .

ii. TransformerEncoderLayer

- **Input dimensions:**

$$(\text{seq_len}, \text{batch_size}, d_{\text{model}})$$

- **Output dimensions:**

$$(\text{seq_len}, \text{batch_size}, d_{\text{model}})$$

- **Equation:**

$$X_1 = \text{LayerNorm}(X + \text{MHA}(X, X, X))$$

$$X_2 = \text{LayerNorm}(X_1 + \text{FFN}(X_1))$$

iii. ViT (Vision Transformer)

- **Input dimensions:**

$$(\text{batch_size}, C, H, W)$$

- **Output dimensions:**

$$(\text{batch_size}, \text{num_classes})$$

- **Equation:**

First apply patch embedding:

$$Z_0 = [x_{\text{class}}; z_1; \dots; z_N] + E_{\text{pos}}$$

Then pass through encoder layers:

$$Z_L = \text{Encoder}(Z_0)$$

Final classification:

$$y = \text{MLP}(Z_L^{(0)})$$

where $Z_L^{(0)}$ is the class token.

4(e) For DINO, we use two ViT systems. The teacher is trained on the full input image. The student is trained on zoomed versions of the same input image. Self distillation with no labels trains the student to produce the same output as the teacher (who is not trained). For each one of the 12 images in the DINO example in the code, comment on high attention values for image segmentation of the two cats. Specifically, highlight image regions that appear:

- i. correct.
- ii. wrong.
- iii. may be correct.

Solution

Below I comment on the 12 DINO attention maps (read left-to-right, top-to-bottom).

1. **Mostly correct:** high attention on the left cat's torso and the right cat's face/neck. **Wrong:** small background spillover on the blanket.
2. **Correct:** attention highlights both cats, especially the right cat's head and the left cat's upper body. **May be correct:** slight attention on nearby blanket folds.
3. **Correct:** strong segmentation of the left cat's full body and part of the right cat's body. **Wrong:** misses some of the right cat's lower body.
4. **May be correct:** both cats are visible, but there is also strong attention on the background and couch, so segmentation is less clean.
5. **Correct:** left cat is well highlighted, and the right cat's head/neck region is identified. **Wrong:** extra attention leaks to the lower-right blanket.
6. **Correct:** strong attention on most of the right cat and much of the left cat. **May be correct:** some attention on the remote and nearby edges.
7. **Correct:** attention captures the left cat and the right cat's head. **Wrong:** some isolated background hotspots.
8. **Correct:** both cats are strongly highlighted, especially their bodies. This is one of the best segmentation maps. **Wrong:** minor spillover to the lower background.
9. **Correct:** attention focuses on the main body regions of both cats. **Wrong:** a few extra bright spots appear off the cats.
10. **Correct:** left cat is segmented well and the right cat is partially segmented. **May be correct:** the remote receives some attention because it is close to the cats.
11. **May be correct:** the map finds parts of both cats, but also attends to larger blanket regions, so the segmentation is less precise.
12. **Correct:** the right cat's face and torso are highlighted, and part of the left cat is also detected. **Wrong:** some body regions are missed, especially on the left cat.

Overall, the best heads are the ones where high attention stays concentrated on the cats' bodies, faces, and legs. The weaker heads are those where attention spreads onto the blanket, couch, or remote. Some regions marked outside the cats may still be *possibly correct* if they lie on boundaries, shadows, or nearby object edges that help define the cats' shapes.

Problem 5. (Contrastive learning models)

For the basics of contrastive learning, refer to Self-supervised contrastive learning with SIMCLR.

For the CLIP problem, refer to: Fixed 16 vision and multimodal transformers.ipynb.

5(a) Based on the example with SIMCLR, for this problem, refer to `contrast_transforms`. Provide pseudocode to expand the list of possible transforms for:

- i. A bigger variety of scalings.
- ii. Different color variations.
- iii. Different blurring functions.

Solution**Algorithm 1** Expanded Contrastive Transforms

```

1: Input: Image  $x$ 
2: Output: Augmented image  $\tilde{x}$ 

3: i. Larger variety of scalings
4: Choose scaling transform  $T_s$  from:
5:   RandomResizedCrop(96, scale = (0.08, 1.0))
6:   RandomResizedCrop(96, scale = (0.2, 0.8))
7:   RandomResizedCrop(96, scale = (0.5, 1.0))
8:   Resize(random size in [80, 140])
9:   RandomCrop(96) or CenterCrop(96)
10: Apply  $x \leftarrow T_s(x)$ 
11: Apply RandomHorizontalFlip( $p = 0.5$ )

12: ii. Different color variations
13: Choose color transform  $T_c$  from:
14:   ColorJitter( $b, c, s, h$ ) with random parameters
15:   RandomGrayscale( $p = 0.2$ )
16:   RandomSolarize(threshold)
17:   RandomPosterize(bits)
18:   RandomAutocontrast()
19:   RandomEqualize()
20: Apply  $x \leftarrow T_c(x)$ 

21: iii. Different blurring functions
22: Choose blur transform  $T_b$  from:
23:   GaussianBlur( $k$ )
24:   MedianBlur( $k$ )
25:   MotionBlur( $k, \theta$ )
26:   BoxBlur( $k$ )
27: Apply  $x \leftarrow T_b(x)$ 

28: Convert to tensor and normalize:
29:  $x \leftarrow \text{ToTensor}(x)$ 
30:  $x \leftarrow \text{Normalize}(x)$ 
31: return  $\tilde{x} = x$ 

```

5(b) Explain how the following transformations may not work for contrastive learning:

- i. Random resizing.
- ii. Random resized cropping.

Solution

- i. **Random resizing:** Random resizing alone may not be effective for contrastive learning because it does not significantly change the semantic content or viewpoint of the image. If two augmented views differ only by scale, the model can rely on low-level features (such as texture or color distribution) to match them, instead of learning meaningful, high-level representations. This weak augmentation reduces the difficulty of the contrastive task and can lead to poor generalization.
- ii. **Random resized cropping:** Random resized cropping can fail if the crops are too aggressive or poorly chosen. In such cases, two crops from the same image may not share sufficient overlapping semantic content (e.g., different parts of an object or entirely different objects in the scene). This creates false positive pairs, where the model is forced to treat unrelated views as similar. As a result, the learned representations may become noisy or less discriminative.

5(c) Evaluate the InfoNCE loss for:

- i. Ideal cases for $N=2$ and $N=1000$, temperature=1.
- ii. Ideal cases for $N=2$ and $N=1000$, temperature=100.
- iii. Ideal cases for $N=2$ and $N=1000$, temperature=0.1.
- iv. Realistic cases for maximum score=0.9, minimum score=0, $N=2$, and $N=1000$, with temperature=1.
- v. Realistic cases for maximum score=0.9, minimum score=0, $N=2$, and $N=1000$, with temperature=0.1.
- vi. Realistic cases for maximum score=0.9, minimum score=0, $N=2$, and $N=1000$, with temperature=100.

Discuss the effect of changing the temperature.

Solution

The InfoNCE loss is

$$\ell_{i,j} = -\log \left(\frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)} \right)$$

For one positive pair and $2N - 2$ negative examples, this becomes

$$\ell = -\frac{s_p}{\tau} + \log(\exp(s_p/\tau) + (2N - 2) \exp(s_n/\tau))$$

where s_p is the positive similarity score, s_n is the negative similarity score, and τ is the temperature.

For the ideal case, use

$$s_p = 1, \quad s_n = -1.$$

Example calculation for $N = 2$ and $\tau = 1$:

$$\ell = -1 + \log(\exp(1) + 2 \exp(-1))$$

$$\ell = -1 + \log(2.718 + 0.736)$$

$$\ell = -1 + \log(3.454) = 0.2395$$

Ideal Cases			
N	τ	s_p	Loss
2	1	1.0	0.2395
1000	1	1.0	5.6036
2	100	1.0	1.0853
1000	100	1.0	7.5804
2	0.1	1.0	0.0000
1000	0.1	1.0	0.0000

Realistic Cases			
N	τ	s_p	Loss
2	1	0.9	0.5951
1000	1	0.9	6.7011
2	0.1	0.9	0.0002
1000	0.1	0.9	0.2204
2	100	0.9	1.0926
1000	100	0.9	7.5914

For the realistic cases,

$$s_p = 0.9, \quad s_n = 0.$$

As the temperature τ decreases, the softmax becomes sharper. This makes the model focus more strongly on the highest similarity scores, so good positive pairs receive much smaller loss. As τ increases, the softmax becomes flatter, meaning the model treats positive and negative examples more similarly. This usually increases the loss and makes contrastive learning less discriminative. Also, as N increases, there are many more negative examples in the denominator, so the loss becomes larger.

5(d) Provide a block diagram showing the architecture for pre-training.

Solution

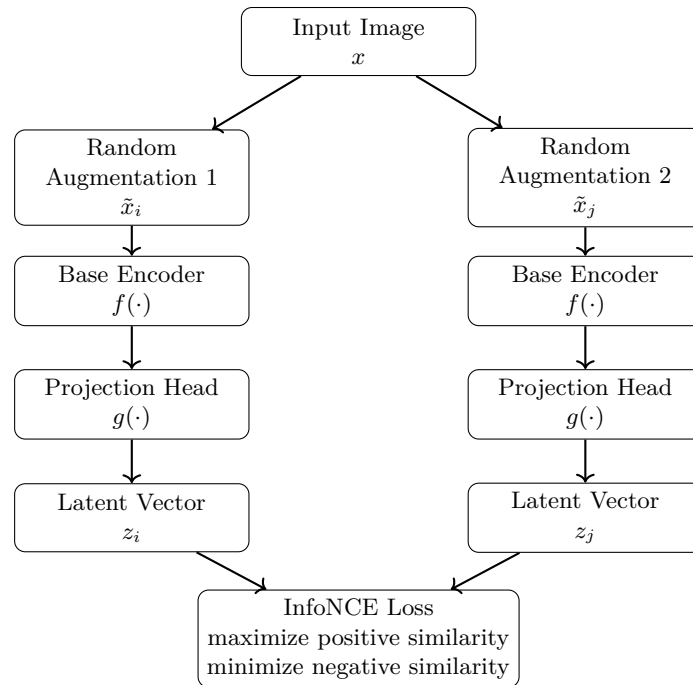


Figure 7: Block diagram of the SimCLR pre-training architecture.

5(e) Provide a block diagram showing the architecture for classification.

Solution

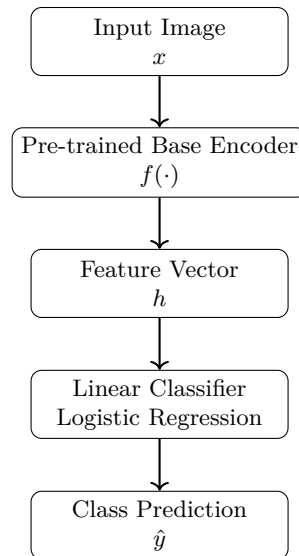


Figure 8: Block diagram of the classification architecture using the pre-trained SimCLR encoder.

5(f) Consider the multimodal transformer based on CLIP. In our example, we have the following code for encoding text and images separately:

```
similarities = image_features @ text_features.T
temperature = clip_model.logit_scale.detach().exp()
rescaled_similarities = similarities * temperature
probabilities = torch.nn.functional.softmax(rescaled_similarities , dim=1)
```

Study the high and low similarity values produced here and answer the following: Explain the following:

- i. What is the ideal similarity when a generated caption corresponds to the image?
- ii. What is the ideal similarity when a generated caption does not correspond to the image?
- iii. What is a realistic similarity when a generated caption corresponds to the image?
- iv. What is a realistic similarity when a generated caption does not correspond to the image?

Solution

- i. **Ideal similarity (matching caption):** The similarity should be as high as possible. Since CLIP uses normalized embeddings and dot products (cosine similarity), the ideal value is close to 1, meaning the image and text embeddings are perfectly aligned.
- ii. **Ideal similarity (non-matching caption):** The similarity should be as low as possible. Ideally, this would be close to -1 , indicating that the embeddings are in opposite directions and completely unrelated.
- iii. **Realistic similarity (matching caption):** In practice, similarities are typically less than perfect. A realistic value for a correct image-caption pair is around 0.7 to 0.9, indicating strong but not perfect alignment due to noise, ambiguity, and imperfect representations.
- iv. **Realistic similarity (non-matching caption):** For incorrect pairs, similarities are usually near 0, meaning the embeddings are roughly orthogonal and share little semantic meaning. Values may range from slightly negative to slightly positive (e.g., -0.2 to 0.2), depending on partial or coincidental feature overlap.